

Parallel and distributed computing

Prof. Dr. ing. Anca Hangan
Department of Computer Science
Anca.Hangan@cs.utcluj.ro

Content - Parallel

- Parallel HW
- Cache coherence on parallel platforms
- Parallel program models
- Parallel algorithms design
- Improving the performance of parallel programs
- Workload-driven evaluation models for parallel programs

Content - Distributed

- Time: clock synchronization, order of events
- Mutual exclusion
- Leader election
- Data processing in IoT Systems

Bibliography

- Parallel Computing

- (Grama) Introduction to Parallel Computing, A. Grama, A. Gupta, G. Karpypis, V. Kumar, 2003
- (Culler) Parallel Computer Architecture: A Hardware / Software Approach, D.E. Culler, J.P. Singh, A. Gupta, Morgan Kaufman Publishers, 1999

- Distributed Computing

- (Kshemkalayani) Distributed Computing: Principles, Algorithms, and Systems, A. D. Kshemkalayani, M. Singhal, Cambridge University Press, 2008
- (Coulouris) Distributed Systems. Concepts and Design, G. Coulouris, J. Dollimore, T. Kindberg, Addison-Wesley, Fifth Edition, 2012

Rules and grading

- Enroll in Moodle(lab, quizzes), MSTeams (code: **8h4jc15**)
- Lab attendance: 100%
- Grading – earn points!
 - Lab assignments + tests: 40p
 - Exam: 50p
 - Course quizzes, attendance:10p
 - To pass the exam: at least 50p (of which at least 25p from exam)

Definitions

- **Parallel** computer: “collection of processing elements that communicate and cooperate to solve **large problems fast**” (Gottlieb et al 1989)
- **Parallel** computing: “a type of computation in which many calculations or the execution of processes are carried out simultaneously” (Gottlieb et al 1989)

Definitions

■ Distributed system

- "one in which several autonomous processors and data stores supporting processes and/or databases interact in order to cooperate to achieve an overall goal. The processes coordinate their activities and exchange information by means of information transfer over a **communication network**." (Sloman and Kramer 1987)
- "a collection of independent computers that appear to the user of the system as a single computer" (Tanenbaum 1995)
- "one in which hardware or software components located at **networked computers** communicate and coordinate their actions only by passing messages" (Coulouris et al 2001)

Definitions

- **Concurrent** system: “one where a computation can advance without waiting for all other computations to complete” (Silberschatz)
- **Concurrent** computations are executed during overlapping time periods

Concurrent, Parallel, Distributed

- **Not equivalent**
- ...but, the terms **overlap**, there is no clear distinction between them
- Concurrent tasks can be executed
 - on a single processor that supports multi-tasking (no parallelism!)
 - on a parallel computer
 - on computers connected through a network

Parallel computing

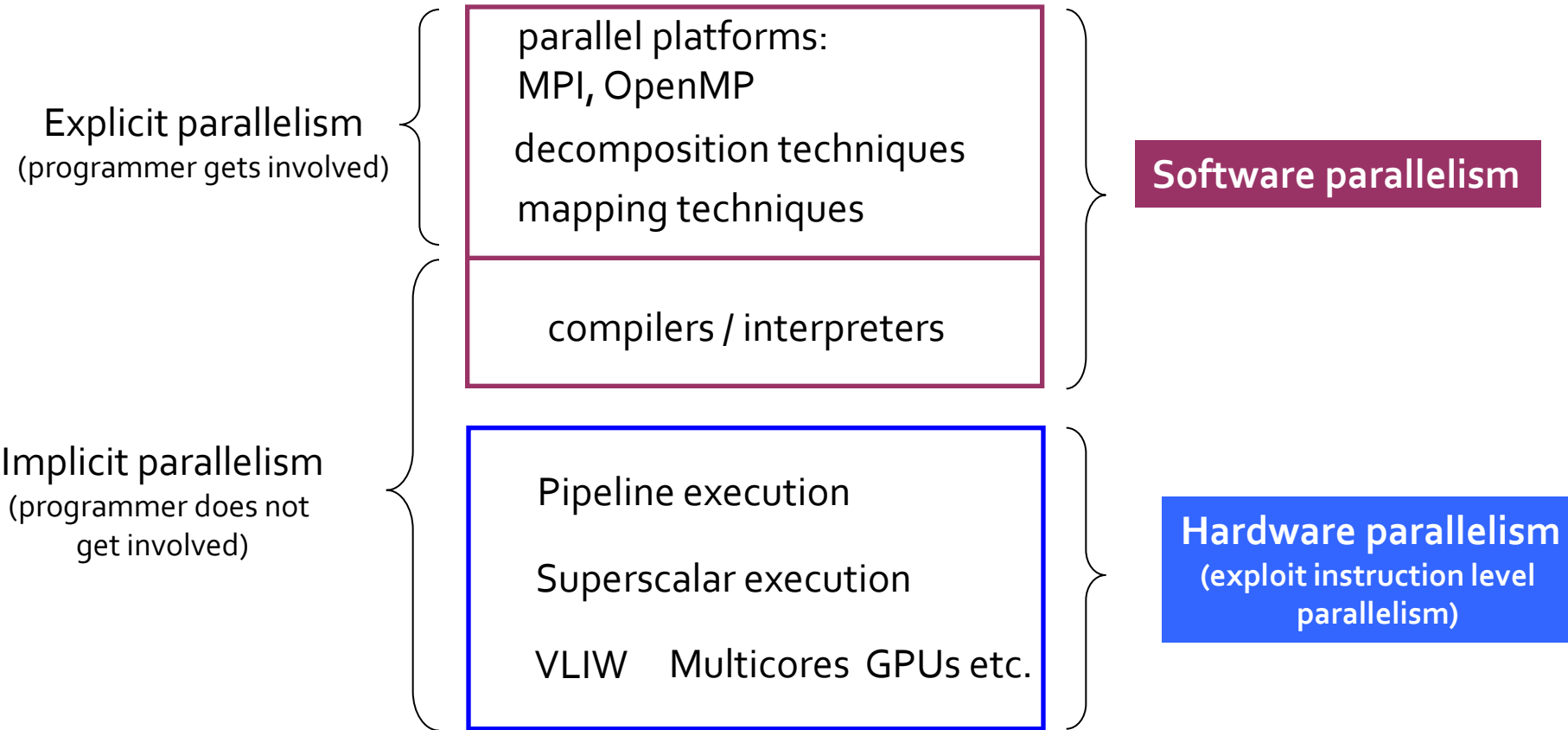
Why parallel computing?

- Solve problems faster
 - ~~Increase processor frequency (Moore's law)~~
 - Pipeline and superscalar processor architectures
 - Multicores / Multiprocessors
 - Accelerators, hybrid HW architectures
- Off-the-shelf parallel computers at a reasonable price -> good reason to write parallel programs; rewrite sequential programs
- Solve really BIG problems in a reasonable time
 - Lots of data
 - Lots of computations
- Get a more accurate result in a reasonable time

Applications of parallel computing

- **Artificial intelligence:** data pre-processing, model training and inference, hyperparameter tuning, ...
- **Computer systems:** image/signal processing, cryptography, security, ...
- **Engineering and design:** mathematical modeling, geometric modeling, ...
- **Scientific applications:** simulations, weather modeling, DNA sequencing, 3D plasma modeling, ...
- **Commercial applications:** transactions, data analysis, ...

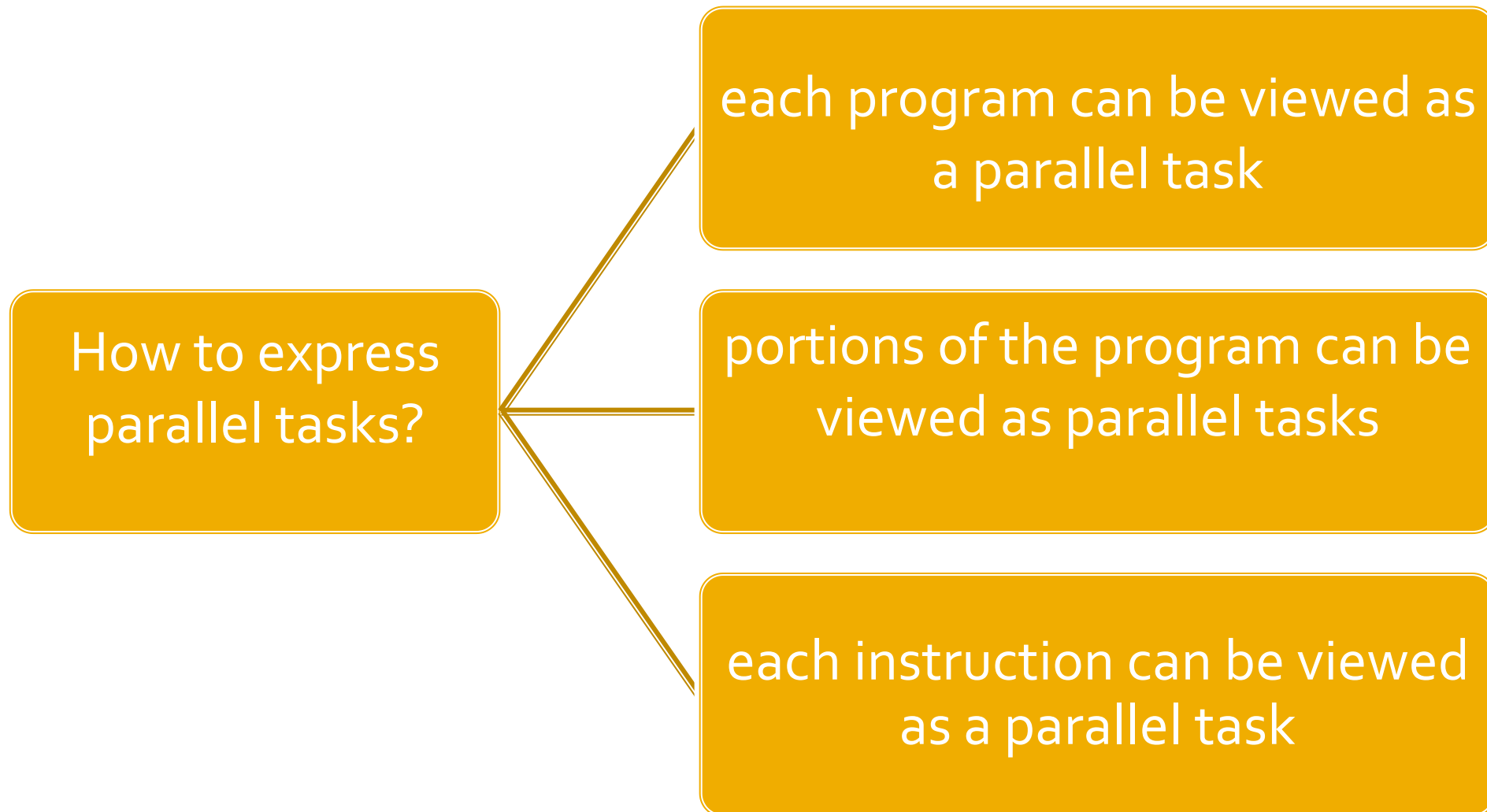
Levels & types of parallelism



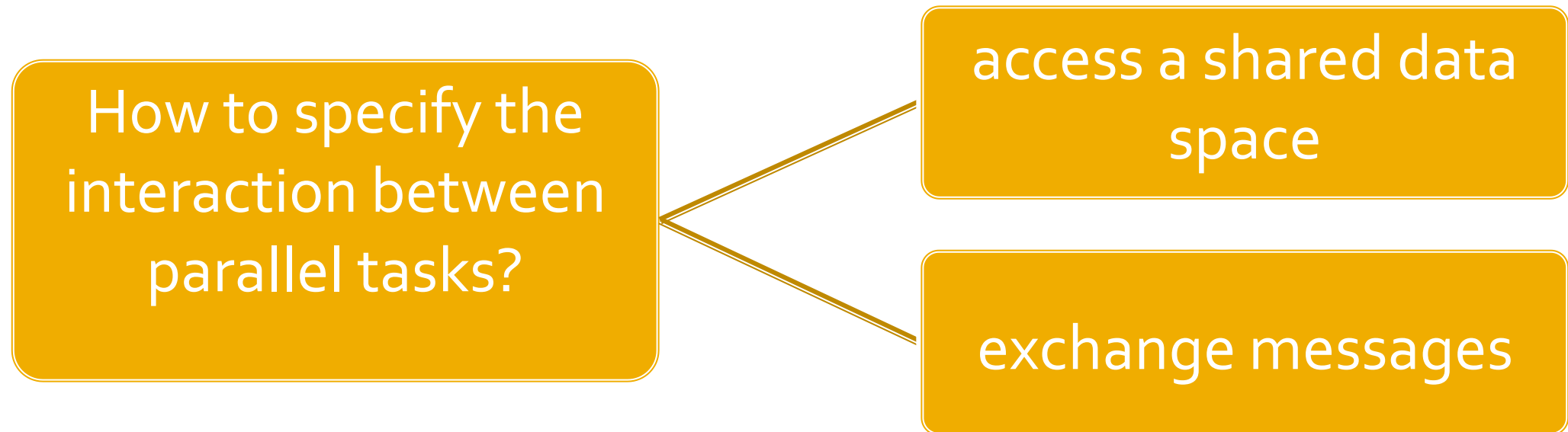
Levels & types of parallelism

- Level of parallelism
 - Program / process level parallelism
 - Thread level parallelism
 - Instruction level parallelism
- Types of parallelism
 - Data parallelism
 - Control (functional) parallelism

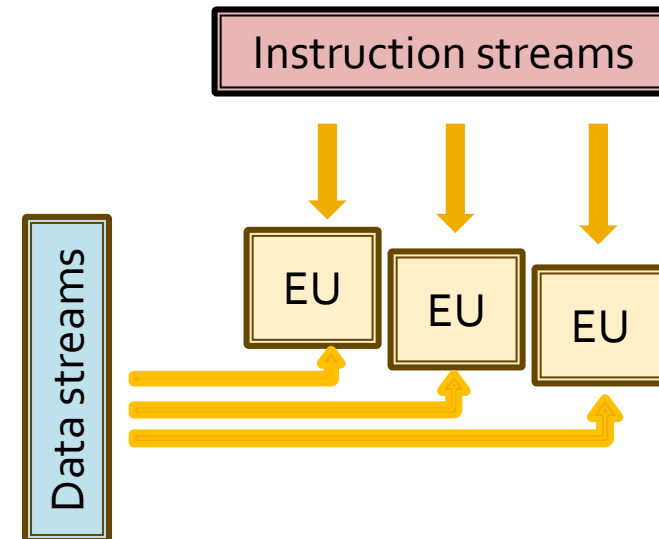
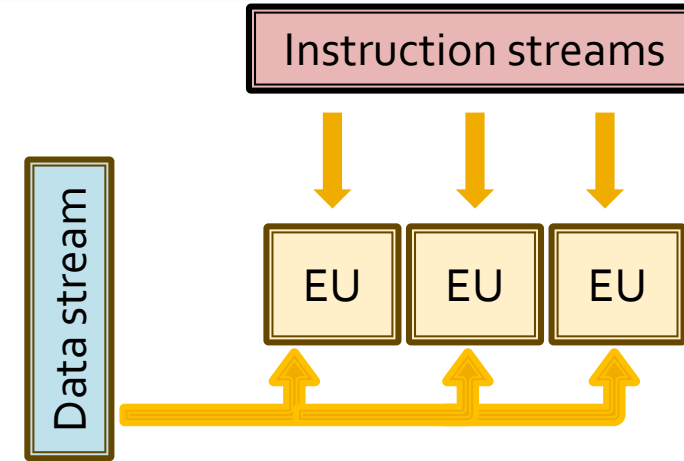
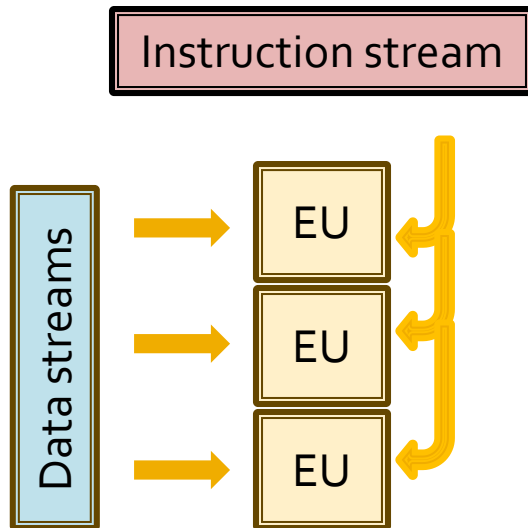
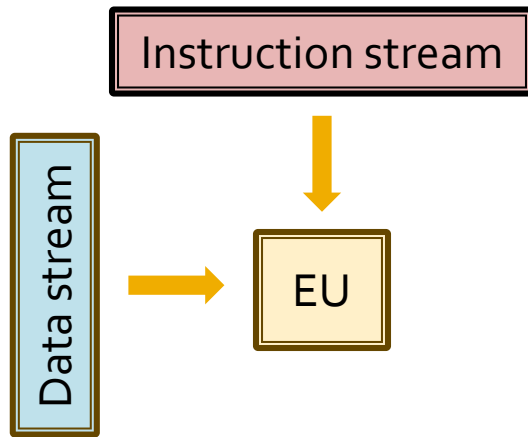
Critical components of parallel computing (programmer's perspective)



Critical components of parallel computing (programmer's perspective)



Parallel architectures classification – Flynn



Distributed computing

Why distributed computing?

Inherently distributed computations e.g. money transfer

Resource sharing: peripherals, data/databases, processing, ...

Access remote resources: sensitive data that cannot be replicated , special (expensive) resources,...

Enhanced reliability: many distributed resources are not likely to crash at the same time

Increased performance/cost ratio: replace special, expensive, parallel machines with distributed computers (ex: Grids).

Examples of distributed systems

- **Computer networks are everywhere and are support for a variety of services!**
- Web search: billions of web pages, billions of searches per month
- Massively multiplayer online games: hundreds of thousands of users, fast response times
- Financial trading: real-time access to information sources, event detection

Models

Architectural models

- how are responsibilities distributed between system components
- how these components are placed

Interaction models

- how to handle the time
- time limits on process execution or message delivery?

Architectural models



Client server

client calls a service on a server (by sending a request message to the server)

server does the work and sends the result back to the client

server can act as a client for other servers

issue: centralization (point of failure, bottleneck)



Peer-to-peer

all processes are equal

every computer holds resources that are commonly shared

no bottleneck for processing and communication

issue: high complexity



Service-oriented

Integrates distributed, separately maintained and deployed software components

Service = functionality that can be accessed remotely

Service consumers are not aware of the services' inner workings

Interaction models

Synchronous

- lower and upper bounds on the execution time of processes
- messages are received within a known bounded time
- drift rates between local clocks have a known bound
- global physical time (with a certain precision)
- predicable behavior in terms of timing (proper for hard real-time apps)
- timeouts can be used to detect failures
- difficult and costly to implement

Asynchronous

- **no** lower and upper bounds on the execution time of processes
- messages are **not** received within a known bounded time
- drift rates between local clocks do **not** have a known bound
- **no** global physical time (logical time is needed)
- **un**predictable in terms of timing
- timeout **cannot** be used for failures detections
- widespread in practice

Finally...

Parallel vs Distributed

- Goal
 - parallel: solve large problems faster, get more accurate results
 - distributed: cooperate to solve a problem (resource sharing, remote access, improve reliability)
- Processors types
 - parallel: identical, non-identical processing units
 - distributed: heterogeneous
- Memory
 - parallel: shared
 - distributed: not shared
- Geographical distribution
 - parallel: close to each other
 - distributed: far from each other